

DSCI 551 Project Proposal

Data Source

For our data source, we decided to go with a clickstream dataset for online shopping (Bojan Tunguz, 2021) from Kaggle. There are 165,474 rows and 14 columns. This data was captured in 2008 and spans across 5 months.

We chose this dataset for a few reasons:

1. It's non-proprietary and easily accessible via the Kaggle API on any computer. It can also be pulled from the [UC Irvine Machine Learning Repository](#) (*UCI Machine Learning Repository*, n.d.). Since it's an already existing dataset that can be requested from an API, we can spend more time building the database and front-end application.
2. There are 14 columns that are a mix of categorical and numerical data. There are also date fields and a unique identifier field. This variety of data will allow us to create a more interactive and engaging application while also fulfilling the requirements of the project.
3. The online shopping data is sourced from a store that offers clothing for pregnant women. This additional context enables us to tell a clearer story with the data and create a more cohesive final application.

Chosen Database

We chose DuckDB because it's a lightweight, open-source database that's really good at running analytics directly on local files like CSVs and Parquet. Instead of being a big server you have to stand up and connect to, it runs in-process (for us, inside a Python app), which makes it easy to plug into a small project like our clickstream dashboard. It's also designed for analytics rather than day-to-day OLTP processes, which lines up with what we actually want to do, which is lots of group-bys, filters, and aggregations on clickstream data.

Internally, DuckDB is interesting because it uses columnar storage and vectorized execution, which we can actually see and talk about in our reports. Columnar storage means it only reads the columns a query needs (for example, country, timestamp, and price), instead of dragging in entire rows every time, which should make our dashboard queries faster on this dataset. The vectorized engine processes data in chunks instead of one row at a time, so operations like scanning, filtering, and aggregating over the clickstream table are done in tight, CPU-friendly loops that we can inspect using **EXPLAIN** and **EXPLAIN ANALYZE**.

DuckDB also fits our workflow nicely. Because it runs inside our app, we don't have to manage a separate database server, users, or network connections just to experiment with queries. We can load the Kaggle clickstream CSV into DuckDB, build dashboard views on top of it, and then connect what the user sees (charts, metrics, etc.) to what DuckDB is doing under the hood in terms of storage layout and query execution.

Planned Internal Focus Areas

1. Columnar Storage: Data is stored in DuckDB in a columnar format that is divided into column segments and row groups. We will examine how column pruning is enabled by columnar storage, which allows DuckDB to read only the attributes a query requires. This is particularly crucial for our dashboard application, where queries usually only access a subset of columns and users can choose from a variety of metrics or filters.
2. Vectorized Query Execution: We will examine how group-by aggregations, joins, filters, and scans are carried out within fixed-size DataChunks as opposed to row-by-row. The chunk-at-a-time execution model is essential for DuckDB to function effectively on medium-sized analytical workloads because dashboards frequently rely on aggregation-heavy queries (counts, averages, grouped summaries, etc.).
3. Data Ingestion: Users can load new CSV or tabular data at runtime because our dashboard is made to support multiple datasets. As a result, we will look at how `read_csv_auto` in DuckDB handles automatic schema inference and type detection as well as how data is transformed into DuckDB's internal format following loading.

Preliminary Application Idea

For our application, we will be creating a self-service customer insights dashboard on Streamlit for a hypothetical online clothing store. This dashboard will be primarily targeted at the operations and finance departments of this hypothetical company. The goal is for the teams to use the dashboard to understand consumer behavior and report any impactful findings to company leadership who want to make data-driven business decisions.

Some initial business questions we want to answer include (but aren't limited to):

1. How popular has the website been overall in recent months?
2. Which products are getting many clicks? Which ones aren't doing well? How does this look over time?
3. Which countries do we generate the most clicks from on average? Are there any that are up-and-coming? How would this influence our marketing campaigns?

4. For the average person, how long do they spend on the site, measured by number of clicks? Is the number of clicks influenced by the initial category they're interested in, the type of photos they see, the location of the photo, or if they see lower prices than average? In other words, does seeing anything in particular cause them to click to the next page?
5. Which category are people exploring the longest? (page number in e-store)

Some initial interactive panels on the dashboard include (but aren't limited to):

- Total/average clicks and unique sessions by month, country, and product type
- Most viewed product categories and products
- Distribution of price by category or location on the page
- Tracking the path of sessions that reach product page and/or those that show interest in higher priced items

Team information and responsibilities

Jesse Fulcher - I'll be responsible for building out the backend of the application using Python and DuckDB, including loading and modeling the clickstream dataset, implementing the main analytical queries that power the dashboard, and wiring up the API or backend logic that the frontend will call to display metrics and visualizations.

Xiaoyi Kuang - I will be responsible for conducting background research to ensure the accuracy and reliability of the information used, clarifying key concepts for better team understanding, and organizing project documentation with proper citations to maintain academic integrity and clarity.

Vivian Tran - I will be responsible for selecting our data source, designing and building the front-end application that uses the data from DuckDB, and ensuring that the final application and any additional analysis of the data is catered to the target end-user.

Initial References

Bojan Tunguz. (2021). *Clickstream Data for Online Shopping*. Kaggle.com.

<https://www.kaggle.com/datasets/tunguz/clickstream-data-for-online-shopping>

GitHub User. (2025). *Documentation*. DuckDB. <https://duckdb.org/docs/stable/>

Streamlit. (2024). *Streamlit Docs*. Docs.streamlit.io. <https://docs.streamlit.io/>

Why DuckDB. (n.d.). DuckDB. https://duckdb.org/why_duckdb